

Extended Abstract

Motivation Imitation learning algorithms usually suffer from distribution shifts as a consequence of compounding errors over time, and generalization errors especially when the environments and/or demonstrations are non-stationary. A key problem is that higher level action abstractions such as ACT (Action Chunking with Transformers) have been demonstrated to decrease distribution shifts, but they fail at non-stationary environments or ones that require precise manipulation. In this paper, we are aiming to augment the action chunking aspect of ACT such that it is able to generalize to more rapidly changing environments, and balance long horizon planning rapid adaptation.

Method We believe that the fixed chunk sizes of the standard ACT implementation (without temporal ensembling) are less effective when the environment requires more fine-grained control by the model. To resolve this, we propose dynamic action chunking, where we introduce an algorithm to determine an appropriate variable chunk length based on the current observation difficulty.

In another direction, we study the temporal ensembling algorithm introduced in ACT. At any timestep, actions from older action chunks may be considered stale when compared to actions from more recent action chunks. Therefore, we implement an outlier detection algorithm to further smooth the temporal ensembling algorithm.

Implementation We trained ACT models on the sim-transfer-cube-human task from the ALOHA dataset. For dynamic action chunking, we compute average Manhattan distance between action outputs of two models trained on different seeds as a proxy for observation difficulty. Based on this, if models agree (small distance) we keep larger action chunk length, and if they disagree (large difference), we decrease the action chunk length by trimming the predicted chunk.

For outlier removal, at every timestep we calculate a sample Mahalanobis distance based on an exponential reweighting of all proposed actions at that timestep. Then, we remove outliers using thresholds set by some multiple of the inter-quartile range (IQR). The exponential reweighting biases the distribution of actions at any timestep towards actions generated from more recent chunks based on the expectation that actions from more recent chunks are advantaged to those from older chunks.

Results Our dynamic action chunking method achieves a 40% success rate compared to the 30% baseline ACT performance without temporal ensembling. The outlier removal method achieves a 56.5% success rate compared to the 45% baseline with temporal ensembling.

For dynamic action chunking, we observe for the first time that the robot is able to properly correct the mistake of missing the cube in its grasp attempt. This evidences the value of providing finer-grain control when unexpected errors are made.

For temporal ensembling, we compare successful and failed rollouts by plotting the Mahalanobis distances from the sample mean at each timestep. We determine that successes are often characterized by the relative agreement of actions generated by different chunks, whereas failures show sharp peaks in the Mahalanobis distances from the sample mean indicating moments of disagreement. We see that outlier removal effectively allows the robot to produce more coherent, consistent averages at each timestep by artificially dampening the amount of disagreement.

Discussion Our implementation still faces various limitations such as the cap on episode length, slow runtime of ACT and extremely sparse reward environment. Additionally, although the metric of model similarity in dynamic action chunking provides good signal, sometimes it can break preplanned motions. Nevertheless, we think this approach holds potential, as we believe successful robotic manipulation algorithms in the future should utilize as much information as possible from their environment to plan optimally while being agile to replanning.

Conclusion We successfully demonstrate that dynamic action chunking and temporal ensembling can improve ACT’s performance in manipulation tasks. The methods address key limitations of fixed chunking strategies and provide principled approaches for handling distribution shift in dynamic environments. Future work should evaluate these approaches in more stochastic real-world scenarios where we expect even greater improvements.

Robust Action Chunking through Dynamic Action Selection

Hristo Todorov

Department of Computer Science
Stanford University
htodorov@stanford.edu

Matthew Li

Department of Computer Science
Stanford University
mattli@stanford.edu

Vivien Cheng

Department of Computer Science
Stanford University
vicheng@stanford.edu

Abstract

Action chunking has emerged as a promising approach for addressing distribution shift in imitation learning for robotic manipulation. However, existing methods like Action Chunking with Transformers (ACT) still suffer from fixed chunk sizes regardless of subtask difficulty. We propose robust action chunking through dynamic action selection to allow for a more flexible policy, introducing two key improvements: (1) adaptive chunk length selection based on model agreement as a proxy for observation difficulty, and (2) outlier-aware temporal ensembling using Mahalanobis distance filtering. Our experiments on the ALOHA cube transfer task demonstrate a 13% improvement in success rate through outlier removal, with analysis revealing clear behavioral differences between successful and failed episodes. These results support our hypothesis that action chunking strategies benefit from adaptability to the fluctuating observation difficulty between various timesteps.

1 Introduction

Fine-manipulation tasks, such as transferring objects or threading cables are notoriously difficult for robots, because they require precise coordination of multiple components, and closed-loop visual feedback. Imitation learning (Pomerleau, 1988) methods have shown promise in such areas, as the sparse reward distribution prevents general reinforcement learning methods from being successful.

However, most imitation learning methods suffer from

- **Distribution shifts.** Small errors compound and lead to unexplored states and therefore the policy makes mistakes (Zhao et al., 2025).
- **Non-stationary environments and demonstrations.** Environments are rapidly changing, and it is difficult to build general policies that adapt to unforeseen circumstances, or model inconsistent human behaviors.

Additionally, tackling one of those two problems amplifies the other: one of the main ways to deal with distribution shifts is through higher-level action representations (Vuong et al., 2025), which makes the policy less adaptive to rapid changes in multiple small timesteps.

One of the most popular approaches employing higher level abstractions to reduce distribution shifts is the to Action Chunking with Transformers (ACT) framework, which uses variational autoencoders

to predict chunks and temporal ensembling to smoothen them (Zhao et al., 2023). However, ACT has notable limitations: it employs fixed action chunk sizes regardless of observation difficulty, and its temporal ensembling mechanism uses simple moving averages without considering the quality or relevance of overlapping action predictions.

In this work, we address these limitations through two complementary contributions: (1) dynamic action chunking that adapts chunk length based on estimated observation difficulty using model agreement, and (2) robust temporal ensembling that filters outlier actions using statistical methods before computing weighted averages. Our approach is motivated by the insight that different subtasks within a manipulation sequence have varying complexity requirements - waypoint navigation may benefit from longer chunks while precise manipulation requires shorter, more responsive chunks. And we believe that successful future approaches in robotics should utilize such context about states to dynamically adjust their planning modes, similar how we humans plan easy versus difficult tasks.

2 Related Work

A traditional way to reduce distribution shifts is to use higher level action abstractions that reduce the horizon and therefore the compounding error accumulation. ACT (Zhao et al., 2023) learns a generative model over action sequences, thus making a higher level action representation which reduces the effective planning horizon and thus reduces compounding errors. Additionally, it utilizes temporal ensembling to balance between outdated older actions and potentially spontaneous and incorrect new actions (Zhao et al., 2023). However, ACT does not incorporate any information about current state when deciding planning horizon (which we believe is a good signal), and the temporal ensembling sometimes fails to smooth the action space. This is the central goal of our paper.

Some higher-level abstraction approaches take into account the current state difficulty. Belkhale et al. (2023)’s HYDRA tackles distribution shifts by decomposing actions into two categories: sparse high-level waypoints (e.g. get to the object) and dense low-level actions (e.g. grab the object). A different policy is trained on each category, and another policy is trained to switch between them. However, HYDRA only has two action abstractions (waypoint and dense), which means that potential higher level action abstractions could not be captured by the waypoint representations. In our method, we try to keep the action abstractions as general as possible (chunks).

Liu et al. (2025)’s Biderctional Decoding (BID) method tries to combine action chunking with closed-loop adaptation to solve the reaction to unexpected changes problem with chunking. They select actions based on two criteria: backward coherence (favors samples that align with previous decisions) and forward contrast (favors samples of high likelihood for future plans), thus promoting both long-term consistency and short-term reactivity. However, their approach does not build intuition for how observation difficulty affects chunk size.

3 Method

3.1 Dynamic Action Chunking

In the standard ACT implementation (without temporal aggregation) (Zhao et al., 2023) the model is trained to predict an action chunk with fixed size k (training hyperparameter), and then it executes those k consecutive actions, gets the updated state, predicts the next k actions etc. A fundamental drawback of this approach is that no matter which state we are currently in, we always predict the next k actions. And although this may work for easier relatively static subtasks where we can plan ahead (e.g. moving to pick up the object), this strategy often breaks when we have more fine-grained or rapidly changing subtasks (e.g. pick the object, when suddenly drop it mid action chunk). Although temporal ACT partially addresses that, it is significantly more computationally expensive at inference time because we sample new action chunk every time steps, so we end up sampling $k \times \text{number of timesteps}$ single actions, which could be inaffordable when we are deploying ACT on long horizon tasks (Zhao et al., 2023).

To address this, we noticed that Belkhale et al. (2023)’s HYDRA algorithm was good at addressing that: it was able to transition between high-level operations (waypoints) and low-level operations (dense manipulation) through training separate policies. To incorporate HYDRA’s insights, we decided to generalize it to the agile action abstractions from ACT (action chunking), which does not

have predefined action mode. We propose an algorithm to determine the appropriate action chunk length through model agreement as metric of observation difficulty.

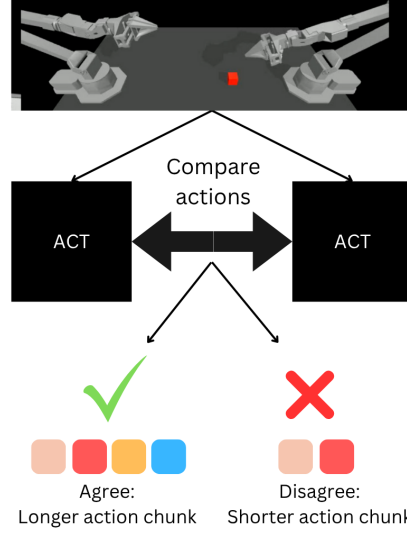


Figure 1: Dynamic action chunking: policy agreement is a good metric of state difficulty, and state difficulty is a good metric of action chunk length.

We train two instances of ACT, each with random seed. At a given timestep t , we sample policy resulting in actions $(a_1, \dots, a_k)$ from the first policy, and $(a'_1, \dots, a'_k)$. To compare how similar they are, we compute the average of the l_1 (Manhattan) distance between each action vectors over the ten different actions

$$d_t = \frac{1}{h} \sum_{i=1}^h \|a_i - a'_i\|_1,$$

where h is the number of future actions across which we are comparing the model outputs (as actions may diverge in the future).

Based on that, we define

$$r = \min\{1, \max\{r_{\min}, \frac{d_t}{\max\{d_1, \dots, d_{t-1}\}}\}\},$$

where $r_{\min}$ is what percentage of the original chunk's size should the new one contain at least, and trim the current chunk size to be of size $c = \lceil r \cdot k \rceil$, so we only take and execute $(a_1, \dots, a_c)$. We use Manhattan distance between each single action vector as the values for the changes in the joint are extremely similar to each other, and their difference is negligible when using Euclidean distance or cosine similarity.

3.2 Outlier Removal

To smooth actions, ACT introduces temporal aggregation, where a new action chunk is predicted at each time step and the action at any given timestep is an exponentially weighted moving average of all previously generated, relevant action chunks. However, we anticipate that environment shifts might lead to stale action chunks, inspiring outlier detection methods (Fig. 2). At each timestep, we collect the distribution of all action chunks and calculate the sample Mahalanobis distance (Kanehiro et al., 2007)

$$d_M(\vec{v}) = \sqrt{(\vec{v} - \mu)^T \Sigma^{-1} (\vec{v} - \mu)} \quad (1)$$

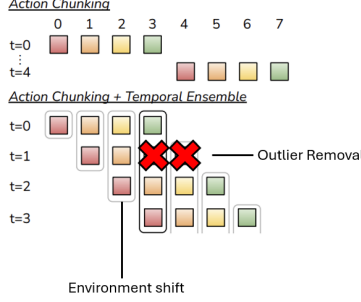


Figure 2: In temporal aggregation, environment shifts might lead to stale actions, leaving outliers which should be removed.

of each action. Here, μ and Σ are the mean and covariance matrix of the distribution respectively. Critically, we make the choice to weight recent actions heavily in an exponential fashion when calculating the sample mean and Mahalanobis distances in the process of determining outliers. Explicitly, for actions

$$\{\vec{a}_1, \dots, \vec{a}_5\} \quad (2)$$

at the same timestep, we calculate Mahalanobis distances using the reweighting

$$\vec{v}_i = e^{-ki} \vec{a}_i, \quad (3)$$

where k is some constant hyperparameter, which we find to work best when it is on the order of 10^{-1} .

Using the Mahalanobis distances we then compute IQR thresholds and remove outlier actions from further consideration. Once outliers are removed at each timestep, we aggregate the remaining actions with a linear average

$$\vec{a}_{pred} = \sum_{i=1}^N \frac{i \vec{a}_i}{\sum_{j=1}^N j} \quad (4)$$

rather than using the ACT paper’s original exponential weighting. Here, N is the number of actions remaining after outlier removal.

To explain the intuition behind our outlier removal method, we now discuss our design process.

The reasoning behind using Mahalanobis distances as a metric relies on our previous experimentation with both the L^2 metric and dimension specific Z-scores as outlier detection methods. Specifically, by weighting based on a covariance matrix, which has off diagonal terms, the Mahalanobis metric can take into account correlations between various action dimensions. To contrast, Z-scores only normalize actions using information of distributions within the same action dimension, and an L^2 metric for calculating the distance between actions completely neglects any different scaling between the different action dimensions.

Before calculating Mahalanobis distances, we decided to weight towards actions from more recently generated chunks because we observed that when the robot reached a timestep which requires a more precise decision, often older action chunks are thrown out. In general, we expect actions from newer chunks to be advantaged to those from older chunks, which may be stale.

We choose to re-weight the remaining actions at each timestep after removing outliers linearly rather than using the original exponential weighting with constant -0.01 proposed by the ACT paper because we found that placing even more bias towards the recently generated action chunks provided a performance increase.

4 Experimental Setup

We study our two methods for improving the performance of action chunking in the transfer cube environment, using the behavior cloning policy and human example data provided in the ALOHA paper. Specifically, in this environment one robotic arm is tasked with lifting a cube from a table, and

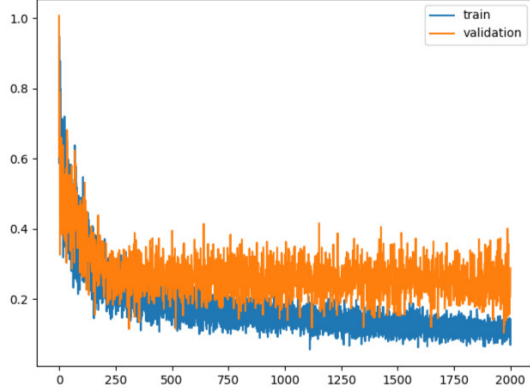


Figure 3: Loss curve from training ACT policy on human data in the sim transfer cube environment

another robotic arm is tasked with receiving the cube. The reward function in this environment is intentionally sparse, with possible rewards of $\{0, 1, 2, 3, 4\}$ at each timestep based on whether certain conditions are met within the environment (e.g. reward for grabbing the cube, higher reward for also lifting it). We trained the base policy with the following hyper-parameters: kl-weight: 10, chunk size: 100, hidden dim: 512, batch size: 8, dim feedforward: 3200, num epochs: 2000, learning rate: $1e-5$. These are the standard hyper parameters given in the ACT source code. Figure 3 shows the learning curve we obtained.

We compare our results relative to baselines produced by the original ACT policy in the transfer cube environment without temporal aggregation for dynamic action chunking and with temporal aggregation for outlier removal.

4.1 Dynamic Action Chunking

We tried out different values for the minimum action chunk size ratio $r_{\min}$, and set $r_{\min} = 20\%$ to ensure that the model will not resort to predicting single actions when unsure, as we have time limit on each episode. Additionally, we set the number of future steps h across which we are comparing $h = 10$ as with larger values the differences would become too big, and with smaller ones they would become too small.

4.2 Outlier Removal

For outlier removal, we did a sweep across various IQR thresholds and weighting constants. We ultimately settled on an IQR threshold of 2.5 and a exponential weighting constant of 0.08.

5 Results

5.1 Quantitative Evaluation

Method	Success Rate
ACT (w/o Temporal Ensembling) (paper)	30%
ACT (w/o Temporal Ensembling) + Dynamic Chunking (ours)	40%
ACT (w/ Temporal Ensembling) (paper)	45%
ACT (w/ Temporal Ensembling) + Outlier Removal (ours)	56.5%

Table 1: Performance Comparison on sim-transfer-cube-human task

Both of our proposed methods show clear improvements over the baseline ACT performance on the cube transfer task. We present results averaged over multiple random seeds in Table 1.

Dynamic Action Chunking Results: Our dynamic action chunking method achieves a 40% success rate compared to the 30% baseline ACT performance without temporal ensembling.

Outlier Removal Results: The outlier removal method achieves a 56.5% success rate compared to the 45% baseline with temporal ensembling.

Computational Cost: Dynamic action chunking requires running two models instead of one, so it roughly doubles both the training and the inference time. However, it is more efficient than ACT + Temporal Ensembling at inference, as it does not calculate new chunk per every timestep.

Outlier removal adds very little computational overhead since the statistical filtering is acting on matrices of relatively small size.

5.2 Qualitative Analysis

In dynamic action chunking, after looking at the values for r (action chunk size ratio) across different timesteps, we noticed that r was high when the model was performing easy waypoint-like tasks (e.g. beginning episode), and much lower when it was performing difficult dense manipulation (e.g. pick up the cube). This means that model agreement was indeed good signal for observation difficulty.

During dynamic action chunking, we noticed through running videos of episodes that while the regular policy will continue performing its planned chunk even if it fails midway (which is intuitive as action chunks are executed end-to-end), in dynamic action chunking our policy will usually soon after it fails (e.g. drops the cube), and try to correct itself. This brought extra performance improvement as usually if the regular ACT policy does not succeed at the first try, the time limit would run out before it could try again.

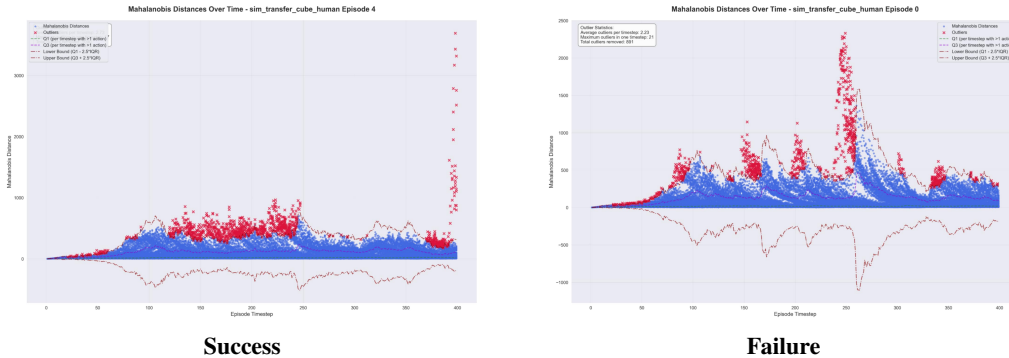


Figure 4: Mahalanobis distances from the sample mean at each timestep in the temporal ensembling process. Red points indicate removed outliers.

For outlier detection, we analyze the factors that make a rollout successful or unsuccessful by considering the deviation of Mahalanobis distances for all proposed actions at each time step from the sample mean in both successful and unsuccessful trials (Fig. 4).

We see that successful episodes often exhibit tightly clustered Mahalanobis distances, which implies consistent predictions among various action chunks. On the other hand, failed episodes show wider spreads in Mahalanobis distances reflecting inconsistent predictions between various action chunks. This indicates that when the model is forced into out of distribution scenarios, various action chunks do not agree on which action to take at any given timestep, leading to an unsuccessful rollout. Specifically, from analysis of the saved videos, we align the peaks in Mahalanobis distance distributions with moments where the first arm misses the cube and tries to correct itself. In these moments which require fine grained manipulation, we expect outlier removal to produce more coherent decision-making by reducing the spread of actions to average over.

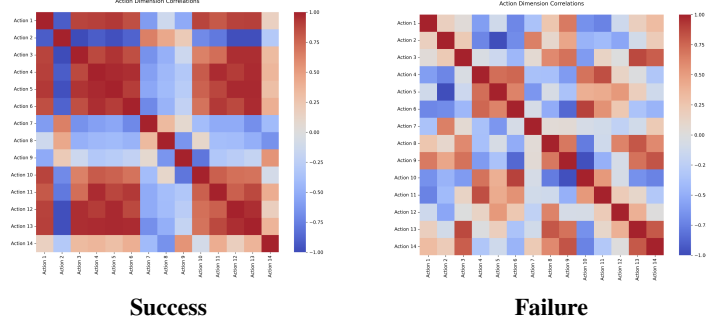


Figure 5: Action-correlation heatmap between all 14 action dimensions, averaged over the entire rollout.

We also observe that in successful episodes, the activations between joints are relatively coherent, while in failed episodes, the activation between various dimensions in the action space is completely incoherent (Fig. 5). Because successful episodes rely on coherence between various action joints, more credence is lent to the importance of using the Mahalanobis metric, as off diagonal terms in the covariance matrix exactly capture this correlation.

6 Discussion

Now we discuss some limitations and comments on our implementation. For dynamic action chunking, sometimes our policy would drop the cube midway and try to pick up the cube again but run out of time before it could be successful. Therefore, one limitation is that the maximum timesteps per episode was set to 400. Nevertheless, this means that there is a lot of room for improvement on how we determine chunk size or select optimal chunks during transition moments, which are usually the trickiest parts of tasks.

For outlier detection, another limitation lies in the assumption that newer action chunks are generally more reliable than older ones, which is a blanket statement and may not always hold true. For instance, sometimes when a distraction is introduced into the environment the robot may want to stay focused on the actions it was originally planning to execute. Therefore, in our implementation we considered estimating the relative advantage of each proposed action individually. However, this approach posed two major challenges. The first is that running inference for each proposed action individually at test time was very slow, as there are around 100 concurrently proposed actions at each time step in the process of temporal ensembling, as our chunk length was 100. The second limitation is that the reward function in the transfer cube environment is extremely sparse, so there was essentially no information to be gained by running inference for each proposed action from each action chunk.

7 Conclusion

Our results demonstrate that both dynamic action chunking and robust temporal ensembling improve the performance of action chunking policies on manipulation tasks. In general, we see that finding a reliable way to quantify observation difficulty provides the backbone for successful adaptive action chunking strategies. Specifically, in the case of dynamic action chunking we measure the L^1 distance between actions proposed by various policies as a proxy for observation difficulty, and in temporal ensembling we use the sample Mahalanobis distances of actions from the same timestep as a proxy.

In future work, we plan to run evaluations in more stochastic environments like the real world, where it is possible to interact with the robot. We expect both outlier removal and dynamic action chunking to show greater improvements from the baseline model in these scenarios, as unexpected environment changes are more common, and our two robustness algorithms allow for increased flexibility of the model in these scenarios.

In another direction, future work can explore a dynamic action chunking strategy that works in tandem with temporal ensembling by using the distribution width of the Mahalanobis distances (Fig.

4 as a proxy for task difficulty in order to better select action chunk length. Because an increase in disagreement between various action chunks at some timestep indicates a difficult task, we can use relative changes in the distribution width as an indicator for ideal dynamic action chunk length.

8 Team Contributions

- **Hristo Todorov:** Implemented the dynamic action chunking algorithm using model agreement between dual ACT instances. Conducted experiments evaluating chunk length adaptation and contributed to overall experimental design and analysis.
- **Matthew Li:** Trained the ACT model to obtain baseline results and model weights. Developed the outlier detection method using the Mahalanobis distance and reweighting of actions. Experimented with K-means for outlier detection. Tried "GRPO inspired" method for outlier detection.
- **Vivien Cheng:** Developed an initial outlier detection method using L^2 metric. Performed detailed statistical analysis of Mahalanobis distance patterns in successful vs. failed episodes, producing visualization figures and correlation heatmaps. Conducted the behavioral analysis connecting joint coordination patterns to success rates. Led the evaluation framework, running multiple evaluation rounds across different random seeds to optimize performance.

Changes from Proposal Our approach remained largely consistent with the original proposal. We successfully implemented both dynamic action chunking and outlier removal components as planned. The main refinement was using model agreement as a more direct proxy for observation difficulty rather than the initially proposed ensemble R-CNN approach, which proved more practical and effective for our experimental setup.

In our proposal, we hinted at a GRPO inspired approach (Shao et al., 2024) for outlier detection. Because GRPO is a train time algorithm and we are trying to make changes at test time in outlier removal, by this, we implied something along the lines of calculating the relative advantage of the actions proposed by different action chunks in the process of temporal ensembling, and reweighting their importance in outlier removal based on the reward that each of these actions would individually produce. However, we found two issues in trying. The first is that running inference for each proposed action individually at test time was very slow, as there are around 100 concurrently proposed actions at each time step in the process of temporal ensembling, as our chunk length was 100. The second, more important issue, is that the reward function in the ACT environment is extremely sparse, so there was essentially no information to be gained by running inference for each proposed action from each action chunk. However, although there are no explicit implementations of this "GRPO" approach, this trial and error lent the idea that we should weight some actions at a timestep heavier than others, leading to our decision to weight actions from recent chunks heavier (in the process of calculating the sample mean). On average, we expect actions from recently generated chunks to be advantaged to actions from older chunks.

References

- Suneel Belkhale, Yuchen Cui, and Dorsa Sadigh. 2023. HYDRA: Hybrid Robot Actions for Imitation Learning. arXiv:2306.17237 [cs.RO] <https://arxiv.org/abs/2306.17237>
- Fumio Kanehiro, Kiyoshi Fujiwara, Hirohisa Hirukawa, Shin'ichiro Nakaoka, and Mitsuharu Morisawa. 2007. Getting up Motion Planning using Mahalanobis Distance. In *Proceedings 2007 IEEE International Conference on Robotics and Automation*. 2540–2545. <https://doi.org/10.1109/ROBOT.2007.363847>
- Yuejiang Liu, Jubayer Ibn Hamid, Annie Xie, Yoonho Lee, Maximilian Du, and Chelsea Finn. 2025. Bidirectional Decoding: Improving Action Chunking via Guided Test-Time Sampling. arXiv:2408.17355 [cs.RO] <https://arxiv.org/abs/2408.17355>
- Dean A. Pomerleau. 1988. ALVINN: an autonomous land vehicle in a neural network. In *Proceedings of the 2nd International Conference on Neural Information Processing Systems (NIPS'88)*. MIT Press, Cambridge, MA, USA, 305–313.

- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300 [cs.CL] <https://arxiv.org/abs/2402.03300>
- An Dinh Vuong, Minh Nhat Vu, Dong An, and Ian Reid. 2025. Action Tokenizer Matters in In-Context Imitation Learning. arXiv:2503.01206 [cs.RO] <https://arxiv.org/abs/2503.01206>
- Michelle Zhao, Reid Simmons, Henny Admoni, Aaditya Ramdas, and Andrea Bajcsy. 2025. Con-formalized Interactive Imitation Learning: Handling Expert Shift and Intermittent Feedback. arXiv:2410.08852 [cs.RO] <https://arxiv.org/abs/2410.08852>
- Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. 2023. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware. arXiv:2304.13705 [cs.RO] <https://arxiv.org/abs/2304.13705>